- When a service requires `NetworkClient`, use an instance of `NetworkClient`.
- When a service requires `MessageFactory`, use an instance of `MessageFactory`.

NOTE You'll also need to register the `EmailServerSettings` object with the DI container. We'll do that slightly differently in section 9.2.

These statements are made by calling various `Add*` methods on the `IServiceCollection` exposed as the `WebApplicationBuilder.Services` property. Each `Add*` method provides three pieces of information to the DI container:

- *Service type*—`TService`. This class or interface will be requested as a dependency. It's often an interface, such as `IEmailSender`, but sometimes a concrete type, such as `NetworkClient` or `MessageFactory`.
- *Implementation type*—`TService` or `TImplementation`. The container should create this class to fulfill the dependency. It must be a concrete type, such as `EmailSender`. It may be the same as the service type, as for `NetworkClient` and `MessageFactory`.
- *Lifetime—transient, singleton, or scoped*. The lifetime defines how long an instance of the service should be used by the DI container. I discuss lifetimes in detail in section 9.4.

DEFINITION A *concrete type* is a type that can be created, such as a standard `class` or `struct`. It contrasts with a type such as an